Universidad Nacional Autónoma de México
Facultad de Ingeniería

Criptografía

Practical Session B – Signing, Encrypting and Hashing Files

Silverio Martínez Andrés

PROFESORA: DRA. Rocío Alejandra Aldeco Pérez

Semestre 2026-1

Grupo: 1

Fecha de entrega: 27 de noviembre de 2025

Practical Session B – Signing, Encrypting and Hashing Files

Using the OpenSSL Command Line Utilities1 do the following tasks:

1. Using the Asymmetric Keys from the previous practical session, sign and verify a text

   file.

2. Encrypt a text file using AES-256.
3. Decrypt a text file using AES-256.
4. Get the hash of a text file using SHA2 of 512 bits.
5. Verify the previous hash.

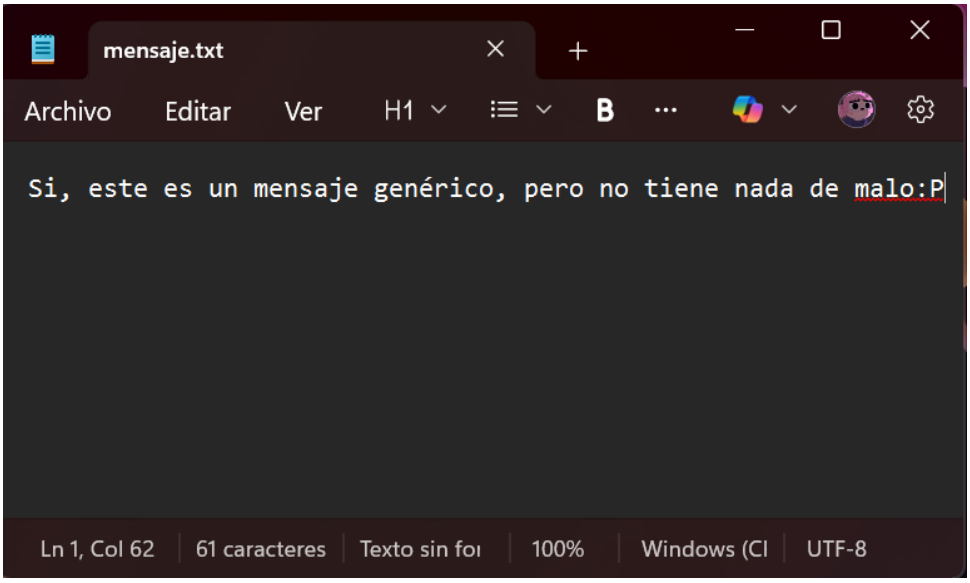
As evidence of these processes create a report that explains which commands you used to achieve each of the tasks, add the corresponding screenshots and explain the process you followed. Add your conclusions and upload it to Canvas.

Desarrollo

Para el inicio de la práctica, se pide, como primer punto:

1. Utilizando las claves asimétricas de la práctica anterior, firma y verifica un archivo de texto

Para ello, utilizaré mi clave privada utilizada para crear una firma digital (firma.sha256) del archivo de texto. Antes de todo, se creó un archivo de texto de nombre “mensaje.txt”, con el siguiente texto.

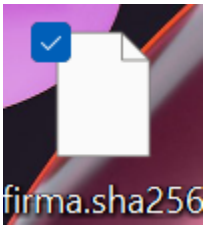


Ahora, para hacer lo que se pide en la práctica, se utiliza el comando “openssl dgst -sha256 -sign llaveprivada.pem -out firma.sha256 mensaje.txt”, donde:

- dgst: Indica que se quiere realizar una operación de resumen de mensaje (hashing), y que con la opción -sign, se convierte en una operación de firma.
- -sha256: Especifica que el algoritmo de hash a utilizar es el SHA-256.
- -sign llaveprivada.pem: Le dice a OpenSSL que calcule el hash y que lo encripte usando la clave privada del archivo “llaveprivada.pem”.
- -out firma.sha256: Define donde se guardará el resultado, que será en un archivo nuevo llamado “firma.sha256”.
- Mensaje.txt: Es el archivo original que se quiere firmar.

Donde, ya corriendo el comando, se genera el siguiente documento:

```
andik@KylosPC MINGW64 ~/Desktop
$ openssl dgst -sha256 -sign llaveprivada.pem -out firma.sha256 mensaje.txt
```



Ahora, para verificar que si se ha creado la firma digital en mi archivo de texto, se necesitará usarl la clave pública igual generada en la práctica anterior. Ahora bien, se utiliza el comando “openssl dgst -sha256 -verify llavepublica.pem -signature firma-sha256 mensaje.txt”, donde:

- -verify llavepublica.pem: Indica a OpenSSL que use la clave pública para intentar desencriptar la firma.
- -signature firma.sha256: Se le indica cual es el archivo que contiene la firma encriptada.

Donde, ya corriendo el comando, se muestra de la siguiente forma la salida en la línea de comandos:

```
andik@KylosPC MINGW64 ~/Desktop
$ openssl dgst -sha256 -verify llavepublica.pem -signature firma.sha256 mensaje.
txt
Verified OK
```

Arrojando el mensaje “Verified OK”, el cual indica que si se ha generado la firma digital y corresponde con el archivo general.

Para el siguiente punto de la práctica:

2. Encripta un archivo de texto utilizando AES-256

Para este paso, se utilizará criptografía simétrica (una contraseña) para proteger el contenido. Se utilizará el comando “openssl enc -aes-256-cbc -salt -in mensaje.txt -out msjencriptado.enc”, donde:

- enc: Indica a OpenSSL que se utilizará el módulo de cifrado simétrico.
- -aes-256-cbc: Indica el estándar de encriptación (AES), los bits de la clave de encriptación (256) y el modo de operación (CBC).
- -salt: Agrega datos aleatorios a la contraseña antes de generar la clave de encriptación, evitando ataques de diccionario.
- -in mensaje.txt: Es el archivo de texto plano que se busca proteger.

Entonces, corriendo el comando, muestra lo siguiente en Git Bash, y genera el documento de texto encriptado:

```
andik@KylosPC MINGW64 ~/Desktop
$ openssl enc -aes-256-cbc -salt -in mensaje.txt -out msjencriptado.enc
enter AES-256-CBC encryption password:

Verifying - enter AES-256-CBC encryption password:

*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

La contraseña que se le puso al documento es: kylos

Además, se nos hace una advertencia que dice que la llave de encriptación utilizada es ambiguo, ya que es el método antiguo que usaba OpenSSL para transformar la contraseña en una llave matemática.

Ahora, paso al punto 3:

3. Desencriptar el archivo con AES-256

Ahora, para ello, se utilizará el comando “openssl enc -d -aes-256-cbc -in msjencriptado.enc -out msjrecuperado.txt”, donde:

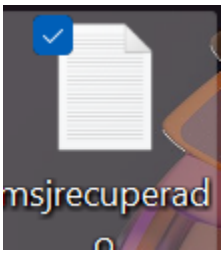
- -d: Esta bandera le dice a OpenSSL que quiere desencriptar los datos.

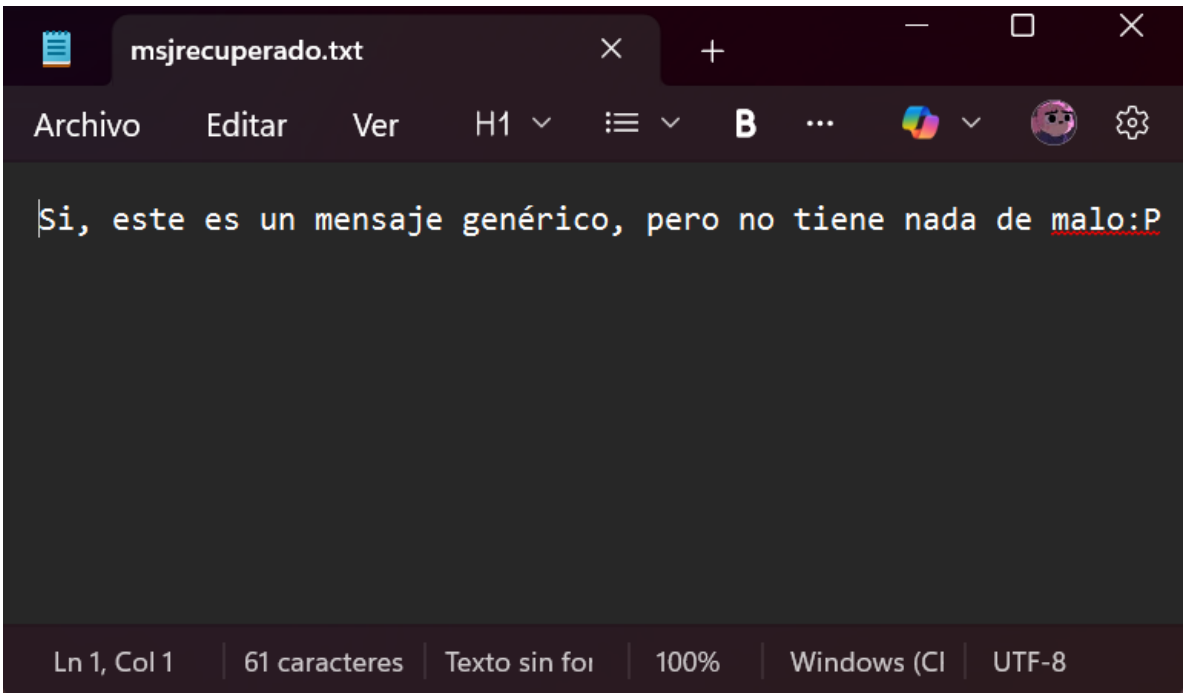
Donde, una vez corriendo el comando, sale lo siguiente:

```
andik@KylosPC MINGW64 ~/desktop
$ openssl enc -d -aes-256-cbc -in msjencriptado.enc -out msjrecuperado.txt
enter AES-256-CBC decryption password:

*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Donde se pide la contraseña puesta en el punto anterior para poder desencriptar el archivo, y que, una vez ya introducida, se generará el archivo “msjrecuperado.txt”, el cual muestra la siguiente información.





Siendo el mismo mensaje que se puso en el archivo “mensaje.txt” del primer punto de la práctica.

Siguiendo con la práctica, el paso 4 pide lo siguiente:

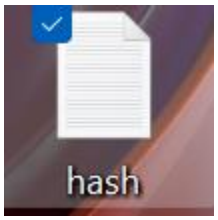
4. Obten el hash de un archivo de texto utilizando SHA2 de 512 bits

Para ello, primero se genera una “huella digital” única dell acrhivo usando el algoritmo SHA-512, esto mediante el comando “openssl dgst -sha512 -out hash.txt mensaje.txt”, donde:

- -sha512: Se utiliza SHA-512 que genera una huella digital de 512 bits.

Donde, corriendo el comando, sale el siguiente resultado:

```
andik@KylosPC MINGW64 ~/desktop
$ openssl dgst -sha512 -out hash.txt mensaje.txt
```



Ya, por último, el final de la práctica pide lo siguiente:

5. Verifica el hash anterior

Donde, en OpenSSL, por lo general, el verificar un hash, por lo general significa comprobar que el archivo no ha sido alterado. Si el archivo cambia un solo bit, el hash será totalmente distinto. Entonces, para verificar el hash del paso anterior, se hacen varios pasos.

a. Mostrar el hash original, esto con el comando “cat hash.txt”

```
andik@KylosPC MINGW64 ~/desktop
$ cat hash.txt
SHA2-512(mensaje.txt)= 38ea37facd35bb633fca9fb0d0dcea47538e75b0fc320857c5a16cb89
a20018ce2c391e16c18bbd463f5c3ab7c7c8c0aa1e886764ffc718a6966d17e696560a1
```

b. Calcular el hash nuevamente, esto mediante el comando “openssl dgst -sha512 mensaje.txt”

```
andik@KylosPC MINGW64 ~/desktop
$ openssl dgst -sha512 mensaje.txt
SHA2-512(mensaje.txt)= 38ea37facd35bb633fca9fb0d0dcea47538e75b0fc320857c5a16cb89
a20018ce2c391e16c18bbd463f5c3ab7c7c8c0aa1e886764ffc718a6966d17e696560a1
```

Donde, si se ve con atención, ambos códigos son los mismos, lo cual confirma que el hash se hizo, donde, si estos códigos no coincidieran, significaría que el archivo “mensaje.txt” fue corrompido o modificado después de la creación de “hash.txt”.

Conclusiones

Con la realización de esta práctica logré integrar técnicamente los conceptos de autenticidad, confidencialidad e integridad mediante el uso de OpenSSL. A través de la firma digital con claves RSA comprobé la eficacia del no repudio, validando la identidad del emisor, mientras que el cifrado simétrico AES-256 me permitió asegurar la confidencialidad de la información haciéndola ilegible sin la contraseña correcta. Finalmente, la generación de resúmenes criptográficos con SHA-512 resultó fundamental para comprender la integridad de los datos, demostrando que el hash actúa como una huella digital inmutable que permite detectar cualquier manipulación o corrupción en los archivos de manera inmediata

Referencias

OpenSSL commands - OpenSSL Documentation. (s. f.).

https://docs.openssl.org/master/man1/#options